UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

DIRECCIÓN GENERAL DE TECNOLOGÍAS DE LA INFORMACIÓN Y COMUNICACIÓN.

FACULTAD DE ESTUDIOS SUPERIORES ARAGÓN.

SERVICIO SOCIAL

Sistema de Adquisición y Registro de Datos

ALUMNO.

Rodríguez Pichardo Adrián Leonardo.

ASESOR.

Dr. José Alberto Aparicio Santos.

SEMANA 4 Y 5:

18 de marzo de 2026 a 31 de marzo de 2026

INDICE

IMPLEMENTACIÓN DEL SISTEMA DE ADQUISICIÓN Y REGISTRO DE DATOS AMBIENTALES MEDIANTE SCRIPTS DE PYTHON 1
1. INTRODUCCIÓN 1
2. ENTORNO DE DESARROLLO Y DEPENDENCIAS 1
3. ARCHIVO CONFIGURACION.PY 2
3.1 CREDENCIALES DE TELEGRAM 2
3.2 CONFIGURACIÓN DE PROMETHEUS 2
3.3 DICCIONARIO DE SENSORES 2
3.4 PARÁMETROS DE HISTORIAL Y ZONA HORARIA 3
3.5 UMBRALES DE ALERTAS 4
4. ARCHIVO CLIENTE_PROMETHEUS.PY 4
4.1 DICCIONARIO LAPSOS 4
4.2 CONSULTA INSTANTÁNEA 5
4.3 CONSULTA DE RANGO 6
4.4 FUNCIONES PUBLICAS 6
5. ARCHIVO GENERADOR_REPORTES.PY 8
5.1 FUNCIÓN AUXILIAR _FORMATO_EJE_X 8
5.2 GENERACIÓN DE IMAGEN PNG 9
5.3 GENERACIÓN DE REPORTE PDF 10
5.4 GENERACIÓN DE ARCHIVO CSV 12
6. ARCHIVO BOT_TELEGRAM.PY 13
6.1 MECANISMO DE SELECCIÓN DE LAPSO CON BOTONES INLINE 13
6.2 COMANDOS DISPONIBLES 14
6.3 HANDLER DE CALLBACKS 16
6.4 VERIFICADOR DE ALERTAS EN SEGUNDO PLANO 17
6.5 INICIALIZACIÓN DEL BOT 18
7. DESPLIEGUE COMO SERVICIO DEL SISTEMA OPERATIVO 19
REFERENCIAS 21

IMPLEMENTACIÓN DEL SISTEMA DE ADQUISICIÓN Y REGISTRO DE DATOS AMBIENTALES MEDIANTE SCRIPTS DE PYTHON

1. INTRODUCCIÓN

Durante esta semana, con la infraestructura de hardware y monitoreo ya operativa desde semanas anteriores, la actividad central consistió en desarrollar un conjunto de scripts en Python que permitieran automatizar la adquisición de datos desde Prometheus, generar reportes en distintos formatos y distribuirlos a través de un bot de Telegram. Para esta semana el producto esperado son los scripts de adquisición, es decir, los archivos de código que implementan toda la lógica de consulta, procesamiento y entrega de información al usuario.

El punto de partida fue la API HTTP que Prometheus expone de forma nativa, la cual permite consultar métricas tanto en su valor instantáneo como en series de tiempo históricas mediante solicitudes GET con parámetros en formato PromQL. A partir de esa interfaz se diseñaron cinco archivos Python con responsabilidades bien delimitadas, siguiendo un principio de separación de responsabilidades que facilita el mantenimiento y la extensión del sistema a futuro.

Los cinco archivos resultantes son configuracion.py, que centraliza todos los parámetros del sistema; cliente_prometheus.py, que implementa la comunicación con la API de Prometheus; generador_reportes.py, que produce los archivos de salida en formato PNG, PDF y CSV; bot_telegram.py, que implementa el bot con el que el usuario interactúa; y requirements.txt, que registra las dependencias de terceros necesarias para ejecutar el sistema.

2. ENTORNO DE DESARROLLO Y DEPENDENCIAS

El desarrollo se realizó en Python 3.11 sobre un sistema Debian GNU/Linux. Se utilizo un entorno virtual creado con el módulo venv para aislar las dependencias del proyecto del resto del sistema, evitando conflictos con otras versiones de librerías instaladas globalmente. Las dependencias del proyecto se registran en el archivo requirements.txt y se instalan con pip dentro del entorno virtual activado.

Las librerías de terceros utilizadas son las siguientes:

- La librería python-telegram-bot en su versión 21.3 provee la interfaz asíncrona para interactuar con la API de Telegram, incluyendo el manejo de comandos, botones inline y envío de archivos.
- La librería apscheduler en su versión 3.10.4 permite programar tareas periódicas que se ejecutan en segundo plano dentro del mismo proceso del bot, utilizada para el verificador de alertas que corre cada cinco minutos.
- La librería requests en su versión 2.32.3 maneja las solicitudes HTTP hacia la API de Prometheus.

- La librería matplotlib en su versión 3.9.1 genera las gráficas de temperatura y humedad como imágenes PNG en memoria.
- La librería reportlab en su versión 4.2.2 construye los documentos PDF con tablas y graficas embebidas.
- Finalmente, pytz en su versión 2024.1 gestiona la conversión de timestamps a la zona horaria local de Ciudad de Mexico.

3. ARCHIVO CONFIGURACION.PY

Este archivo actúa como el único punto de configuración del sistema. Todas las variables que pueden necesitar ajuste según el entorno de despliegue, como credenciales, direcciones de red o umbrales de alerta, están agrupadas aquí. El resto de los módulos las importan desde este archivo en lugar de tener valores escritos directamente en su propio código, lo que significa que adaptar el sistema a un entorno diferente solo requiere editar este archivo.

3.1 CREDENCIALES DE TELEGRAM

Las variables TELEGRAM_TOKEN y TELEGRAM_CHAT_ID almacenan respectivamente el token de autenticación del bot y el identificador numérico del chat destino. El token se obtiene al crear el bot mediante BotFather en Telegram y es la clave con la que la librería python-telegram-bot se autentica ante la API de Telegram para enviar y recibir mensajes. El CHAT_ID identifica al destinatario de los mensajes automáticos de alerta; en este caso es el grupo llamado “Reportes DHT22 - Python”.

```
# --- Telegram ---
TELEGRAM_TOKEN   = "8707579631:AAGAsPDbN19y_ypB8pmuWDvxFpxtE_WD28c"
TELEGRAM_CHAT_ID = "5287582092"
```

3.2 CONFIGURACIÓN DE PROMETHEUS

La variable PROMETHEUS_URL define la dirección base del servidor Prometheus. Como Prometheus corre en la misma máquina que los scripts, el valor es http://localhost:9090. Si en algún momento se quisiera mover el bot a una maquina diferente, este sería el único valor a cambiar para que el sistema siga consultando correctamente.

```
# --- Prometheus ---
PROMETHEUS_URL = "http://localhost:9090"
```

3.3 DICCIONARIO DE SENSORES

El diccionario SENSORES es la estructura más importante del archivo de configuración. Cada clave del diccionario es el nombre descriptivo de un sensor y su valor es otro diccionario con todos los parámetros necesarios para identificarlo y consultarlo. Estos parámetros son el nombre del job de Prometheus, la dirección IP y puerto del microcontrolador, el color hexadecimal que se usara en las gráficas para ese sensor, los valores exactos de los labels sensor y node que el firmware utiliza en cada métrica, y los nombres completos de las dos métricas que expone: temperatura y humedad.

El hecho de que cada sensor tenga sus propios nombres de métrica con sufijo diferente (_n para el NodeMCU y _e para el ESP32-C3) fue una decisión de diseño del firmware que se tomó antes del desarrollo de los scripts. Los firmwares fueron desarrollados de manera que las métricas del NodeMCU V3 se llaman dht22_temperature_celsius_n y dht22_humidity_percent_n, mientras que las del ESP32-C3 se llaman dht22_temperature_celsius_e y dht22_humidity_percent_e. Almacenar estos nombres en la configuración en lugar de inferirlos dinámicamente garantiza que los scripts funcionen correctamente aunque los nombres de las métricas no sigan un patrón uniforme.

```
# --- Sensores ---
SENSORES = {
    "NodeMCU V3": {
        "job":          "dht22_sensor_n",
        "instance":     "10.203.58.115:80",
        "color":        "#FF6B6B",
        "label_sensor": "dht22",
        "label_node":   "nodemcu_v3",
        "metrica_temp": "dht22_temperature_celsius_n",
        "metrica_hum":  "dht22_humidity_percent_n",
    },
    "ESP32-C3": {
        "job":          "dht22_sensor_e",
        "instance":     "10.203.58.133:80",
        "color":        "#4ECDC4",
        "label_sensor": "dht22",
        "label_node":   "esp32c3_mini",
        "metrica_temp": "dht22_temperature_celsius_e",
        "metrica_hum":  "dht22_humidity_percent_e",
    },
}
```

3.4 PARÁMETROS DE HISTORIAL Y ZONA HORARIA

La variable HORAS_HISTORIAL define el número de horas de historial que se usa como valor de respaldo cuando no se especifica un lapso concreto. PASO_CONSULTA define la resolución de muestreo por defecto. ZONA_HORARIA se establece como America/Mexico_City para que todos los timestamps generados por el sistema se expresen en hora local en lugar de UTC, lo cual es fundamental para que las gráficas y los reportes muestren horarios comprensibles para el usuario.

```
# --- Parametros de historial ---
HORAS_HISTORIAL = 6
PASO_CONSULTA   = "1m"
ZONA_HORARIA    = "America/Mexico_City"
```

3.5 UMBRALES DE ALERTAS

Las cuatro variables de umbral definen los limites fuera de los cuales el bot envía una notificación de alerta.

- La temperatura máxima se fija en 40 grados Celsius, la mínima en 5 grados.
- La humedad máxima en 85 por ciento y la mínima en 20 por ciento.

Estos valores se eligieron considerando las condiciones ambientales normales de una sala de servidores o espacio de oficina. La variable booleana ALERTAS_ACTIVAS controla si el verificador de alertas debe actuar cuando detecta valores fuera de rango, y puede cambiarse en tiempo de ejecución mediante el comando /alertas del bot.

```
# --- Umbrales de alertas ---
ALERTA_TEMP_MAX  = 40.0
ALERTA_TEMP_MIN  = 5.0
ALERTA_HUM_MAX   = 85.0
ALERTA_HUM_MIN   = 20.0
ALERTAS_ACTIVAS  = True
```

4. ARCHIVO CLIENTE_PROMETHEUS.PY

Este módulo implementa toda la lógica de comunicación con Prometheus. Su responsabilidad es abstraer los detalles de PromQL y de la API HTTP de Prometheus de manera que el resto del sistema pueda obtener datos con llamadas simples a funciones en Python sin necesidad de construir queries manualmente.

4.1 DICCIONARIO LAPSOS

Al inicio del archivo se define el diccionario LAPSOS, que mapea once claves de texto cortas a tuplas de tres elementos. La primera semana de trabajo con el sistema mostro que es útil poder consultar tanto periodos muy cortos, como los últimos cinco minutos para revisar lecturas recientes, como periodos largos, como la última semana o el último mes para analizar tendencias. Por eso se definieron once opciones que cubren ese rango: 5m, 15m, 30m, 1h, 3h, 6h, 12h, 24h, 2d, 1w y 1mo.

Para cada lapso, además de la etiqueta legible que se muestra en Telegram, se almacena la duración en minutos y el paso de muestreo óptimo para ese lapso. El paso de muestreo es el parámetro step que se envía a Prometheus en las consultas de rango y determina cada cuanto tiempo se devuelve un punto de datos. Para lapsos cortos se usa un paso

pequeño, como quince segundos para los últimos cinco minutos, para obtener alta resolución. Para lapsos largos se usa un paso grande, como dos horas para el mes completo, para evitar que Prometheus devuelva decenas de miles de puntos que saturarían la memoria y alentarían la generación del reporte.

```python
# ----------------------------------------------------------------
# Lapsos de tiempo disponibles para consulta
# clave -> (etiqueta legible, minutos totales, paso Prometheus)
# ----------------------------------------------------------------
LAPSOS = {
    "5m":   ("Últimos 5 minutos",   5,       "15s"),
    "15m":  ("Últimos 15 minutos",  15,      "15s"),
    "30m":  ("Últimos 30 minutos",  30,      "30s"),
    "1h":   ("Última hora",         60,      "1m"),
    "3h":   ("Últimas 3 horas",     180,     "1m"),
    "6h":   ("Últimas 6 horas",     360,     "2m"),
    "12h":  ("Últimas 12 horas",    720,     "5m"),
    "24h":  ("Últimas 24 horas",    1440,    "5m"),
    "2d":   ("Últimos 2 días",      2880,    "10m"),
    "1w":   ("Última semana",       10080,   "30m"),
    "1mo":  ("Último mes",          43200,   "2h"),
}
```

4.2 CONSULTA INSTANTÁNEA

La función privada _consulta_instantánea recibe el nombre de la métrica, el valor del label node y el valor del label sensor, y construye una consulta PromQL del formato métrica con los labels entre llaves. Esta consulta se envía al endpoint /api/v1/query de Prometheus mediante una solicitud GET con un timeout de cinco segundos para evitar que una demora en la respuesta bloquee el bot indefinidamente. Si la consulta devuelve resultados, extrae el valor numérico del primer elemento y lo devuelve como flotante. Si no hay resultados o si ocurre cualquier excepción, devuelve None, lo que los módulos superiores interpretan como que el sensor no tiene datos disponibles.

```python
def _consulta_instantanea(metrica: str, node: str, sensor: str) -> float | None:
    """Devuelve el valor actual de una metrica filtrando por node y sensor."""
    query = f'{metrica}{{sensor="{sensor}",node="{node}"}}'
    try:
        r = requests.get(
            f"{PROMETHEUS_URL}/api/v1/query",
            params={"query": query},
            timeout=5,
        )
```

```python
        r.raise_for_status()
        datos = r.json()
        resultados = datos.get("data", {}).get("result", [])
        if resultados:
            return float(resultados[0]["value"][1])
        return None
    except Exception as e:
        print(f"[Prometheus] Error consulta instantanea ({metrica}): {e}")
        return None
```

4.3 CONSULTA DE RANGO

La función privada _consulta_rango extiende la lógica anterior para obtener series de tiempo. Calcula el timestamp de inicio restando los minutos del lapso al momento actual y envía ambos junto con el paso de muestreo al endpoint /api/v1/query_range. La respuesta de Prometheus es una lista de pares timestamp-valor que la función convierte en dos listas paralelas de objetos datetime y valores flotantes respectivamente. Esta estructura de dos listas paralelas es la que luego usa matplotlib para graficar la serie temporal con los tiempos en el eje X y los valores en el eje Y.

```python
def _consulta_rango(metrica: str, node: str, sensor: str,
                    minutos_atras: int, paso: str):
    """
    Consulta una serie de tiempo (range query).
    Devuelve (timestamps[], valores[]) o ([], []).
    """
    tz     = pytz.timezone(ZONA_HORARIA)
    fin    = datetime.now(tz)
    inicio = fin - timedelta(minutes=minutos_atras)
```

```python
    query = f'{metrica}{{sensor="{sensor}",node="{node}"}}'
    try:
        r = requests.get(
            f"{PROMETHEUS_URL}/api/v1/query_range",
            params={
                "query": query,
                "start": inicio.timestamp(),
                "end":   fin.timestamp(),
                "step":  paso,
            },
            timeout=10,
        )
```

```python
        r.raise_for_status()
        datos = r.json()
        resultados = datos.get("data", {}).get("result", [])
        if resultados:
            valores    = resultados[0]["values"]
            timestamps = [datetime.fromtimestamp(float(v[0]), tz) for v in valores]
            lecturas   = [float(v[1]) for v in valores]
            return timestamps, lecturas
        return [], []
    except Exception as e:
        print(f"[Prometheus] Error consulta rango ({metrica}): {e}")
        return [], []
```

4.4 FUNCIONES PUBLICAS

La función obtener_lecturas_actuales itera sobre el diccionario SENSORES e invoca _consulta_instantanea para la métrica de temperatura y para la de humedad de cada sensor. Devuelve un diccionario con el nombre de cada sensor como clave y un sub-diccionario con los campos temperatura y humedad como valor, junto con el timestamp del momento de la consulta. Esta es la función que el bot llama cuando el usuario ejecuta el comando /estado.

```python
def obtener_lecturas_actuales() -> dict:
    """
    Devuelve los valores actuales de todos los sensores.
    {
        "NodeMCU V3": {"temperatura": 25.3, "humedad": 60.1},
        "ESP32-C3":   {"temperatura": 24.8, "humedad": 58.7},
        "timestamp":  datetime
    }
    """
    datos = {"timestamp": datetime.now(pytz.timezone(ZONA_HORARIA))}
    for nombre, cfg in SENSORES.items():
        temp = _consulta_instantanea(
            cfg["metrica_temp"], cfg["label_node"], cfg["label_sensor"]
        )
        hum = _consulta_instantanea(
            cfg["metrica_hum"], cfg["label_node"], cfg["label_sensor"]
        )
        datos[nombre] = {"temperatura": temp, "humedad": hum}
    return datos
```

La función obtener_historial acepta la clave de un lapso como argumento, verifica que sea válida y en caso de que no lo sea la reemplaza por el valor por defecto de seis horas. Extrae la duración en minutos y el paso del diccionario LAPSOS y llama a _consulta_rango para cada sensor y cada métrica. Además del historial de cada sensor, el diccionario de retorno incluye un campo lapso_info con la clave, la etiqueta y los minutos del lapso consultado. Este campo lo utilizan el generador de reportes y el exportador para incluir el periodo correcto en los títulos y nombres de archivo.

```python
def obtener_historial(clave_lapso: str = "6h") -> dict:
    """
    Devuelve el historial para el lapso indicado.
    clave_lapso debe ser una de las claves de LAPSOS (p.ej. '1h', '24h').
    {
        "NodeMCU V3": {
            "tiempos_temp": [...], "valores_temp": [...],
            "tiempos_hum":  [...], "valores_hum":  [...],
        },
        "lapso_info": {"clave": "6h", "etiqueta": "Últimas 6 horas", "minutos": 360}
    }
    """
```

```python
    if clave_lapso not in LAPSOS:
        clave_lapso = "6h"

    etiqueta, minutos, paso = LAPSOS[clave_lapso]

    historial = {
        "lapso_info": {
            "clave":    clave_lapso,
            "etiqueta": etiqueta,
            "minutos":  minutos,
        }
    }
```

```python
    for nombre, cfg in SENSORES.items():
        tt, tv = _consulta_rango(
            cfg["metrica_temp"], cfg["label_node"], cfg["label_sensor"],
            minutos, paso
        )
        ht, hv = _consulta_rango(
            cfg["metrica_hum"], cfg["label_node"], cfg["label_sensor"],
            minutos, paso
        )
        historial[nombre] = {
            "tiempos_temp": tt, "valores_temp": tv,
            "tiempos_hum":  ht, "valores_hum":  hv,
        }

    return historial
```

5. ARCHIVO GENERADOR_REPORTES.PY

Este módulo concentra toda la lógica de generación de contenido. Contiene tres funciones públicas que producen respectivamente una imagen PNG con las gráficas de tendencia, un archivo PDF con el reporte completo, y un archivo CSV con los registros en texto plano. Las tres funciones reciben como argumentos el diccionario de lecturas actuales y el diccionario de historial devueltos por el módulo anterior.

5.1 FUNCIÓN AUXILIAR _FORMATO_EJE_X

Antes de las funciones públicas se define una función auxiliar privada que recibe un objeto de eje de matplotlib y la duración del lapso en minutos, y configura el formateador y el localizador del eje X de forma adaptativa.

- Para lapsos de hasta una hora usa formato HH:MM con marcas cada cierto número de minutos.
- Para lapsos de hasta un día usa formato HH:MM con marcas horarias.
- Para lapsos de hasta una semana usa formato dd/mm HH:MM con marcas cada doce horas.
- Para el mes completo usa formato dd/mm con marcas diarias.

Sin esta lógica adaptativa, el eje X de una gráfica de un mes mostraría cientos de marcas ilegibles si se usara el formato de minutos, o aparecería completamente vacío si el localizador horario intentara colocar 720 marcas en el espacio disponible.

```python
def _formato_eje_x(ax, minutos: int):
    """Ajusta el formateador del eje X segun el lapso."""
    if minutos <= 60:
        fmt   = mdates.DateFormatter("%H:%M")
        loc   = mdates.MinuteLocator(interval=max(1, minutos // 6))
    elif minutos <= 1440:
        fmt   = mdates.DateFormatter("%H:%M")
        loc   = mdates.HourLocator(interval=max(1, minutos // 360))
```

```
    elif minutos <= 10080:
        fmt   = mdates.DateFormatter("%d/%m %H:%M")
        loc   = mdates.HourLocator(interval=12)
    else:
        fmt   = mdates.DateFormatter("%d/%m")
        loc   = mdates.DayLocator(interval=max(1, minutos // (60*24*6)))
    ax.xaxis.set_major_formatter(fmt)
    ax.xaxis.set_major_locator(loc)
    plt.setp(ax.xaxis.get_majorticklabels(), rotation=30, ha="right")
```

5.2 GENERACIÓN DE IMAGEN PNG

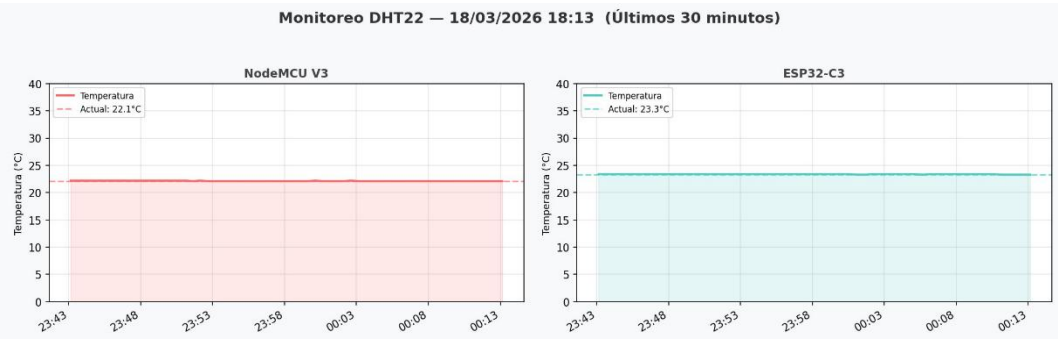
La función generar_imagen_grafica crea una figura de matplotlib con cuatro subplots organizados en una cuadricula de dos filas y dos columnas. La primera columna corresponde al NodeMCU V3 y la segunda al ESP32-C3. La fila superior muestra temperatura y la fila inferior humedad. Esta disposición permite comparar visualmente el comportamiento de ambos sensores en la misma grafica.

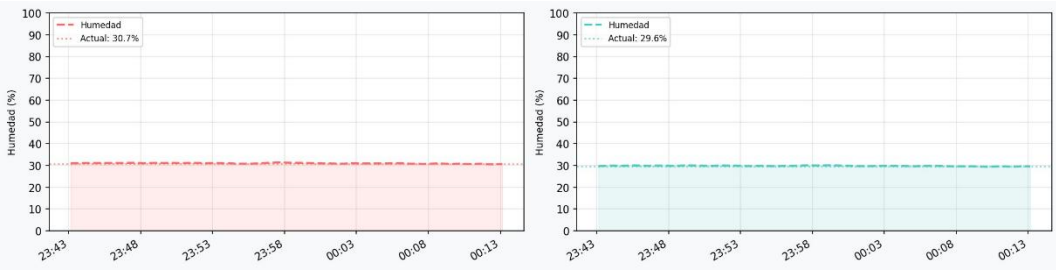
En cada subplot se traza la serie histórica como una línea con relleno semitransparente debajo de ella, lo que facilita percibir la magnitud de los valores además de su tendencia. Sobre la serie histórica se superpone una línea horizontal discontinua que indica el valor actual del sensor en ese momento, permitiendo al usuario ver de un vistazo si el valor presente se encuentra dentro del rango habitual del periodo visualizado.

Los ejes Y están fijados de forma permanente: la temperatura siempre va de 0 a 40 grados Celsius con marcas cada cinco grados, y la humedad siempre va de 0 a 100 por ciento con marcas cada diez puntos. Esta decisión garantiza que dos graficas de periodos distintos sean directamente comparables porque la escala es siempre la misma, algo que no ocurriría si matplotlib ajustara automáticamente los límites del eje Y al rango de los datos de cada consulta.

```
# Ejes fijos para todas las graficas
EJE_TEMP_MIN = 0
EJE_TEMP_MAX = 40
EJE_HUM_MIN  = 0
EJE_HUM_MAX  = 100
```

La figura se guarda en memoria como un objeto BytesIO en formato PNG a 150 DPI y se devuelve sin escribir ningún archivo temporal en disco. Este objeto puede enviarse directamente a Telegram o pasarse a la función de generación de PDF para incluirse en el documento.





5.3 GENERACIÓN DE REPORTE PDF

La función generar_reporte_pdf utiliza la librería reportlab para construir un documento PDF de varias secciones. Primero llama internamente a generar_imagen_grafica para obtener la gráfica y la guarda en un archivo temporal en disco, ya que reportlab necesita una ruta de archivo para insertar imágenes, no puede leerlas desde un objeto en memoria directamente.

La primera sección del PDF es el encabezado con:

- El título del reporte.
- La fecha y hora de generación.
- La etiqueta del periodo consultado.

Reporte de Sensores DHT22

Generado: 18/03/2026 18:13:57 | Periodo: Últimas 3 horas

La segunda sección es una tabla de lecturas actuales con cuatro columnas:

- nombre del sensor.
- temperatura.
- humedad.
- estado.

Lecturas Actuales

Sensor	Temperatura (°C)	Humedad (%)	Estado
NodeMCU V3	22.10	30.70	Normal
ESP32-C3	23.30	29.70	Normal

El estado se calcula comparando los valores actuales contra los umbrales de configuración y puede ser Normal, Alto o Bajo.

La tercera sección es una tabla de estadísticas del periodo con siete columnas:

- sensor.

- métrica.
- valor mínimo.
- valor máximo.
- Promedio.
- desviación estándar.
- numero de registros.

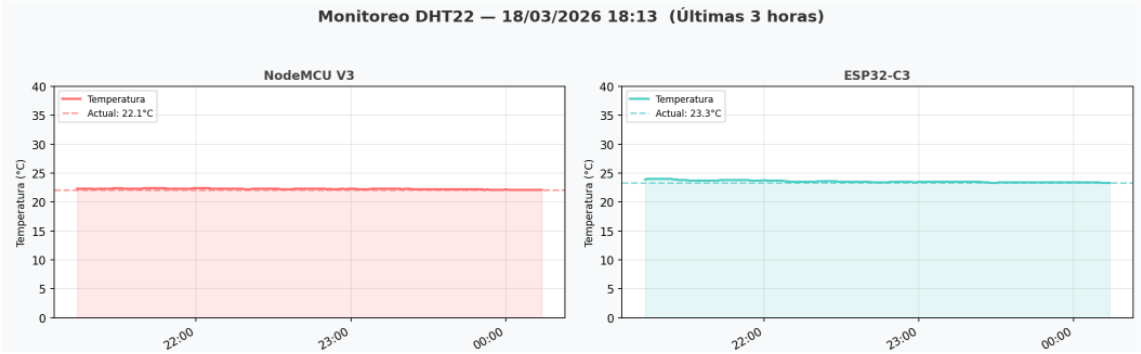
La desviación estándar se calcula con el módulo statistics de la biblioteca estándar de Python y se muestra como N/A cuando el periodo contiene un solo punto de datos.

Estadisticas — Últimas 3 horas

Sensor	Metrica	Minimo	Maximo	Promedio	Desv. Est.	Registros
NodeMCU V3	Temperatura	22.10 C	22.40 C	22.26 C	0.0835	181
NodeMCU V3	Humedad	29.80 %	32.10 %	31.04 %	0.3877	181
ESP32-C3	Temperatura	23.30 C	24.00 C	23.56 C	0.1784	180
ESP32-C3	Humedad	28.40 %	30.10 %	29.54 %	0.3399	180

La cuarta sección inserta la gráfica PNG como imagen.

Graficas de Tendencia



La quinta sección, que comienza en una nueva página, contiene tablas con todos los registros individuales del periodo agrupados por sensor. Cada tabla muestra:

- Un numero de registro correlativo.
- El timestamp exacto.
- La temperatura de ese punto.
- La humedad de ese punto.

Registros Completos — Últimas 3 horas

Todos los puntos de datos capturados por Prometheus en el periodo seleccionado.

Sensor: **NodeMCU V3** — 181 registros

#	Timestamp	Temp. (°C)	Humedad (%)
1	2026-03-18 15:13:57	22.30	30.60
2	2026-03-18 15:14:57	22.30	31.00
3	2026-03-18 15:15:57	22.30	31.10
4	2026-03-18 15:16:57	22.30	31.30
5	2026-03-18 15:17:57	22.30	31.50

Sensor: **ESP32-C3** — 180 registros

#	Timestamp	Temp. (°C)	Humedad (%)
1	2026-03-18 15:13:57	23.90	28.50
2	2026-03-18 15:14:57	24.00	28.70
3	2026-03-18 15:15:57	24.00	28.80
4	2026-03-18 15:16:57	24.00	29.10
5	2026-03-18 15:17:57	24.00	29.30

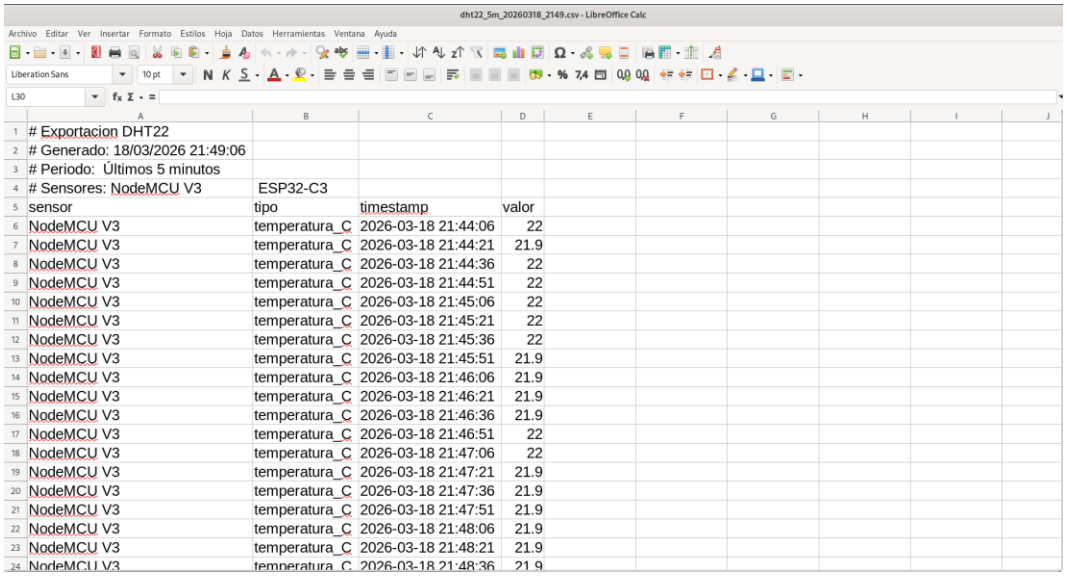
Los encabezados de cada tabla se repiten al inicio de cada nueva página gracias al parámetro repeatRows de reportlab, lo que facilita la lectura cuando la tabla ocupa múltiples hojas.

Para alinear los registros de temperatura y humedad por timestamp se construye un diccionario donde cada clave es la representación textual del timestamp y cada valor es un sub-diccionario con los campos temperatura y humedad. Esto es necesario porque Prometheus puede devolver timestamps ligeramente distintos para la serie de temperatura y la de humedad del mismo sensor si el paso de muestreo no es exactamente divisible, y sin esta alineación las filas de la tabla quedarían desincronizadas.

5.4 GENERACIÓN DE ARCHIVO CSV

La función generar_csv construye un archivo de texto en formato CSV directamente en memoria como un objeto BytesIO. Las primeras cinco líneas son comentarios que comienzan con el carácter # e incluyen la fecha de generación y el periodo de los datos. A continuación se escribe la línea de encabezado con los nombres de las cuatro columnas:

- sensor.
- tipo.
- timestamp.
- valor.



	A	B	C	D
1	# Exportacion DHT22			
2	# Generado: 18/03/2026 21:49:06			
3	# Periodo: Últimos 5 minutos			
4	# Sensores: NodeMCU V3	ESP32-C3		
5	sensor	tipo	timestamp	valor
6	NodeMCU V3	temperatura_C	2026-03-18 21:44:06	22
7	NodeMCU V3	temperatura_C	2026-03-18 21:44:21	21.9
8	NodeMCU V3	temperatura_C	2026-03-18 21:44:36	22
9	NodeMCU V3	temperatura_C	2026-03-18 21:44:51	22
10	NodeMCU V3	temperatura_C	2026-03-18 21:45:06	22
11	NodeMCU V3	temperatura_C	2026-03-18 21:45:21	22
12	NodeMCU V3	temperatura_C	2026-03-18 21:45:36	22
13	NodeMCU V3	temperatura_C	2026-03-18 21:45:51	21.9
14	NodeMCU V3	temperatura_C	2026-03-18 21:46:06	21.9
15	NodeMCU V3	temperatura_C	2026-03-18 21:46:21	21.9
16	NodeMCU V3	temperatura_C	2026-03-18 21:46:36	21.9
17	NodeMCU V3	temperatura_C	2026-03-18 21:46:51	22
18	NodeMCU V3	temperatura_C	2026-03-18 21:47:06	22
19	NodeMCU V3	temperatura_C	2026-03-18 21:47:21	21.9
20	NodeMCU V3	temperatura_C	2026-03-18 21:47:36	21.9
21	NodeMCU V3	temperatura_C	2026-03-18 21:47:51	21.9
22	NodeMCU V3	temperatura_C	2026-03-18 21:48:06	21.9
23	NodeMCU V3	temperatura_C	2026-03-18 21:48:21	21.9
24	NodeMCU V3	temperatura_C	2026-03-18 21:48:36	21.9

Las filas de datos se generan iterando sobre todos los sensores y, para cada uno, sobre las dos series de datos: primero todos los registros de temperatura con el tipo temperatura_C, luego todos los de humedad con el tipo humedad_pct. Los timestamps se formatean en el patrón YYYY-MM-DD HH:MM:SS que es reconocido automáticamente por Excel, pandas y la función readtable de MATLAB sin necesidad de especificar el formato manualmente. Los valores numéricos se expresan con cuatro decimales.

El nombre del archivo resultante incluye la clave del lapso y el timestamp de generación, por ejemplo dht22_6h_20260318_1347.csv, para que el usuario pueda identificar fácilmente el contenido de cada archivo descargado sin necesidad de abrirlo.

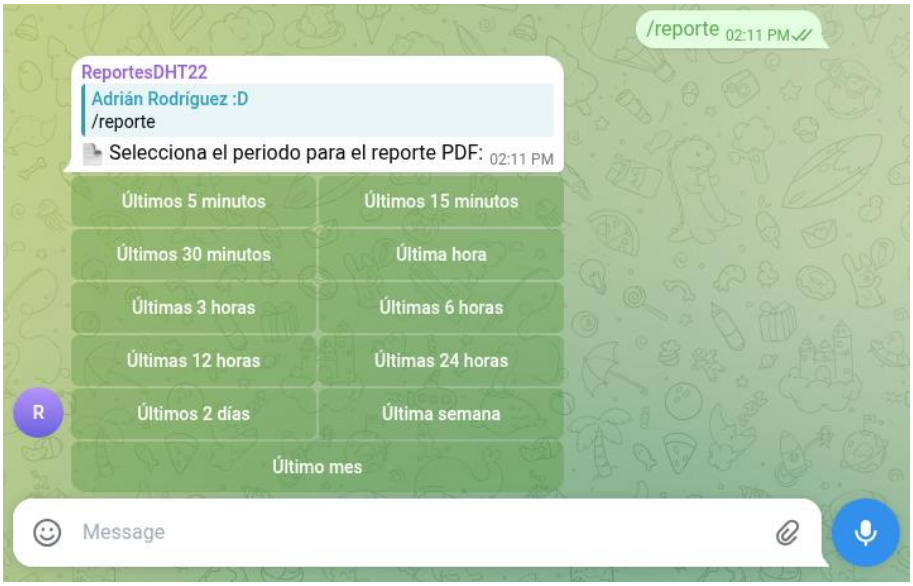
6. ARCHIVO BOT_TELEGRAM.PY

Este es el archivo principal del sistema. Implementa el bot de Telegram que sirve como interfaz de usuario para todo el sistema de monitoreo. Utiliza la librería python-telegram-bot versión 21, que trabaja de forma completamente asíncrona mediante el framework asyncio de Python, lo que significa que el bot puede atender múltiples solicitudes simultaneas y ejecutar tareas en segundo plano sin bloquear la atención de comandos.

6.1 MECANISMO DE SELECCIÓN DE LAPSO CON BOTONES INLINE

Una de las decisiones de diseño más importantes del bot fue implementar la selección del periodo de tiempo mediante botones inline de Telegram en lugar de argumentos de texto en los comandos. Los botones inline son elementos interactivos que aparecen directamente dentro del chat, debajo de un mensaje, y que el usuario puede tocar sin necesidad de escribir nada. Esto elimina la necesidad de recordar la sintaxis exacta de los comandos y hace que la interacción sea fluida desde un dispositivo móvil.

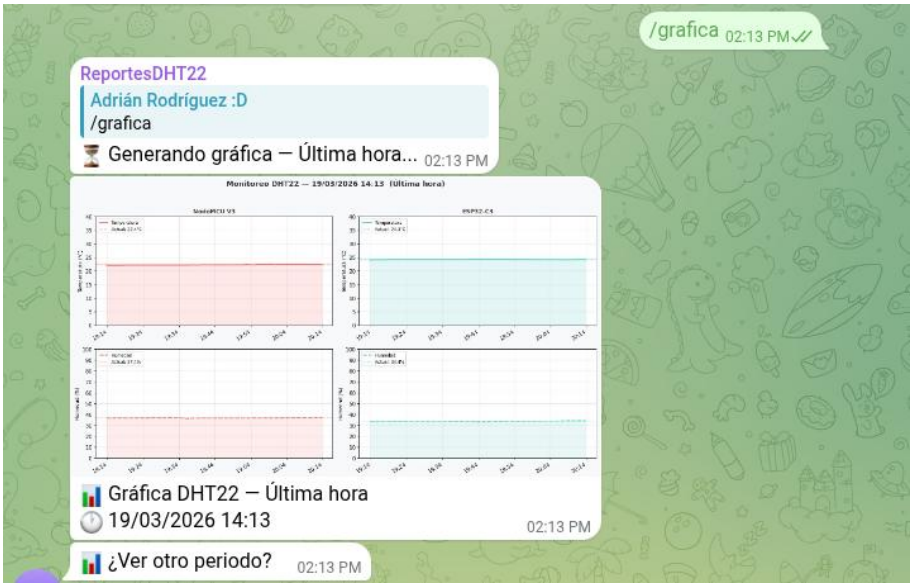
La función _teclado_lapsos genera dinámicamente un objeto InlineKeyboardMarkup iterando sobre el diccionario LAPSOS importado del módulo cliente_prometheus. Cada botón tiene como etiqueta visible la descripción legible del lapso y como callback_data una cadena formada por un prefijo que identifica el tipo de reporte seguido de la clave del lapso, por ejemplo gráfica:6h. Al estar generado dinámicamente a partir del mismo diccionario que define los lapsos disponibles, el teclado se actualiza automáticamente si se agrega o elimina un lapso del diccionario sin necesidad de modificar el código del bot.

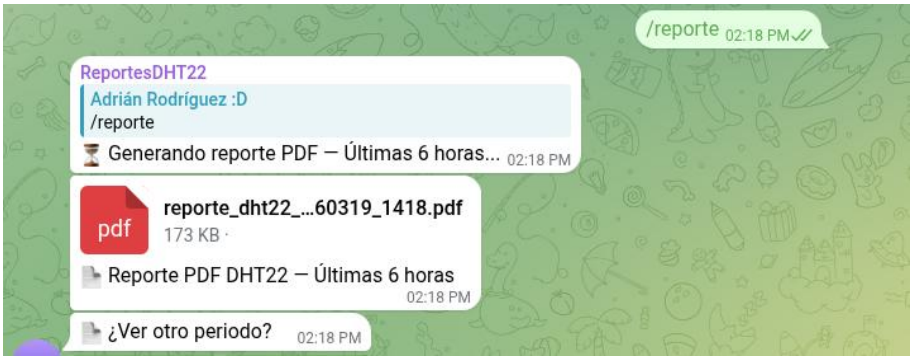


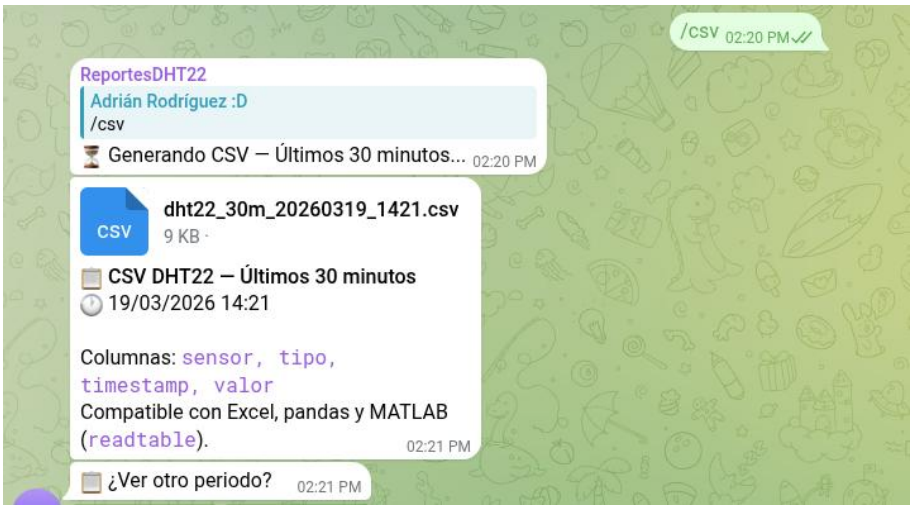
6.2 COMANDOS DISPONIBLES

El comando **/estado** consulta las lecturas actuales llamando a obtener_lecturas_actuales y las formatea en un mensaje de texto con Markdown, mostrando el nombre de cada sensor, su temperatura, su humedad y un emoji de color que indica si los valores están dentro del rango normal, por encima del máximo o por debajo del mínimo.

Los comandos **/grafica**, **/reporte** y **/csv** no generan ningún archivo inmediatamente. En su lugar, envían al usuario el teclado de once botones para que seleccione el periodo de interés. Esto evita que el usuario tenga que esperar el tiempo de procesamiento sin saber qué periodo va a recibir, y le da control explicito sobre el rango de datos que quiere visualizar.







El comando **/alertas** alterna el estado de la variable global alertas_activas entre verdadero y falso, y confirma el cambio al usuario. Esto permite al operador desactivar temporalmente las notificaciones de alerta, por ejemplo durante un mantenimiento planificado en el que se esperan valores fuera de rango, sin necesidad de detener el servicio ni editar el archivo de configuración.

6.3 HANDLER DE CALLBACKS

La función callback_lapso es el handler que procesa todos los toques en los botones inline. Cuando el usuario toca un botón, Telegram envía al bot un objeto CallbackQuery cuyo campo data contiene la cadena definida en el callback_data del botón. El handler extrae el prefijo para determinar qué tipo de archivo debe generar y la parte restante de la cadena para saber el lapso.

El handler responde primero con query.answer() para quitar el indicador de carga del botón y dar retroalimentación inmediata al usuario. Luego actualiza el texto del mensaje de selección con un mensaje de progreso usando query.edit_message_text, lo que evita que queden mensajes obsoletos en el chat. A continuación llama a la función de envío correspondiente, que puede tardar varios segundos dependiendo del lapso seleccionado y la cantidad de datos a procesar. Finalmente, una vez enviado el archivo, envía un nuevo teclado de selección para que el usuario pueda pedir otro periodo sin necesidad de volver a escribir el comando. Este ciclo de selección continua es lo que da al bot la sensación de una interfaz fluida y explorable.

```python
async def callback_lapso(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query   = update.callback_query
    await query.answer()
    data    = query.data
    chat_id = query.message.chat_id

    if data.startswith(PRE_GRAFICA):
        clave = data[len(PRE_GRAFICA):]
        etiq  = LAPSOS.get(clave, ("?",))[0]
        await query.edit_message_text(f"⏳ Generando gráfica — {etiq}...")
        await _enviar_grafica(ctx.application, chat_id, clave)
        await ctx.application.bot.send_message(
            chat_id=chat_id,
            text="📊 ¿Ver otro periodo?",
            reply_markup=_teclado_lapsos(PRE_GRAFICA),
        )
```

```python
    elif data.startswith(PRE_REPORTE):
        clave = data[len(PRE_REPORTE):]
        etiq  = LAPSOS.get(clave, ("?",))[0]
        await query.edit_message_text(f"⏳ Generando reporte PDF — {etiq}...")
        await _enviar_pdf(ctx.application, chat_id, clave)
        await ctx.application.bot.send_message(
            chat_id=chat_id,
            text="📄 ¿Ver otro periodo?",
            reply_markup=_teclado_lapsos(PRE_REPORTE),
        )

    elif data.startswith(PRE_CSV):
        clave = data[len(PRE_CSV):]
        etiq  = LAPSOS.get(clave, ("?",))[0]
        await query.edit_message_text(f"⏳ Generando CSV — {etiq}...")
        await _enviar_csv(ctx.application, chat_id, clave)
        await ctx.application.bot.send_message(
            chat_id=chat_id,
            text="📋 ¿Ver otro periodo?",
            reply_markup=_teclado_lapsos(PRE_CSV),
        )
```

6.4 VERIFICADOR DE ALERTAS EN SEGUNDO PLANO

La función verificacion_alertas se registra como un job de APScheduler con un CronTrigger que la ejecuta cada cinco minutos. Esta función llama a obtener_lecturas_actuales, compara cada valor contra los cuatro umbrales configurados y, si alguno esta fuera de rango, construye y envía un mensaje de alerta al TELEGRAM_CHAT_ID. La función respeta el estado de la variable alertas_activas y no hace nada si las alertas están desactivadas.

APScheduler se integra con el loop de asyncio de python-telegram-bot mediante el AsyncIOScheduler, que ejecuta los jobs como corrutinas dentro del mismo loop de eventos que atiende los comandos del bot. Esto significa que el verificador de alertas y el bot comparten el mismo proceso y el mismo loop sin necesidad de hilos adicionales ni procesos separados, lo que simplifica el despliegue y la gestión de recursos.

6.5 INICIALIZACIÓN DEL BOT

En la función main se construye la aplicación de Telegram con el token del archivo de configuración, se registran todos los handlers de comandos y el handler de callbacks, y se define la función post_init que actualiza el menú de comandos visible en Telegram al arrancar el bot. Este menú es el que aparece cuando el usuario escribe una barra inclinada en el chat y muestra la lista de comandos disponibles con su descripción.

```python
# ================================================================
# Main
# ================================================================

def main():
    app = Application.builder().token(TELEGRAM_TOKEN).build()

    app.add_handler(CommandHandler("start",   cmd_start))
    app.add_handler(CommandHandler("estado",  cmd_estado))
    app.add_handler(CommandHandler("grafica", cmd_grafica))
    app.add_handler(CommandHandler("reporte", cmd_reporte))
    app.add_handler(CommandHandler("csv",     cmd_csv))
    app.add_handler(CommandHandler("alertas", cmd_alertas))
    app.add_handler(CommandHandler("ayuda",   cmd_ayuda))
    app.add_handler(CallbackQueryHandler(callback_lapso))

    async def post_init(application: Application):
        await application.bot.set_my_commands([
            BotCommand("estado",  "Lecturas actuales de los sensores"),
            BotCommand("grafica", "Gráfica PNG — elija el periodo"),
            BotCommand("reporte", "Reporte PDF completo — elija el periodo"),
            BotCommand("csv",     "Exportar datos en CSV — elija el periodo"),
            BotCommand("alertas", "Activar/desactivar alertas automáticas"),
            BotCommand("ayuda",   "Mostrar comandos"),
        ])

    app.post_init = post_init
```

El scheduler se configura con la zona horaria de Ciudad de Mexico y se arranca antes de iniciar el polling. La llamada a run_polling con drop_pending_updates en verdadero descarta cualquier mensaje o comando que el usuario haya enviado mientras el bot estaba apagado, evitando que se procesen solicitudes antiguas al reiniciar el servicio.

```python
    scheduler = AsyncIOScheduler(timezone=ZONA_HORARIA)
    scheduler.add_job(
        verificacion_alertas,
        trigger=CronTrigger(minute="*/5", timezone=ZONA_HORARIA),
        args=[app],
        id="chequeo_alertas",
    )
```

```python
    scheduler.start()
    log.info("Scheduler iniciado. Bot corriendo...")
    app.run_polling(drop_pending_updates=True)


if __name__ == "__main__":
    main()
```

7. DESPLIEGUE COMO SERVICIO DEL SISTEMA OPERATIVO

Una vez verificado el funcionamiento correcto de todos los scripts ejecutándolos directamente desde la terminal, el paso final de la semana fue configurar el bot para que se ejecute de forma permanente como un servicio del sistema operativo, sin necesidad de mantener una sesión de terminal activa ni un editor de código abierto.

En Debian GNU/Linux la herramienta estándar para gestionar servicios es systemd, que es el sistema de inicialización adoptado por la mayoría de las distribuciones Linux modernas. Systemd permite definir procesos como unidades de servicio mediante archivos de texto con extensión .service ubicados en el directorio /etc/systemd/system/. Cuando systemd gestiona un proceso, se encarga de iniciarlo al arrancar el sistema, reiniciarlo automáticamente si falla, y capturar su salida estándar en el journal del sistema para su consulta posterior.

Se creo el archivo */etc/systemd/system/dht22bot.service* con las directivas necesarias:

- La directiva **After** con el valor *network-online.target* garantiza que el servicio no intente iniciarse hasta que la conexión de red esté disponible, lo que es fundamental para que el bot pueda autenticarse ante la API de Telegram desde el primer intento.
- La directiva **User** especifica que el proceso debe correr bajo mi usuario *"adrian"* en lugar del superusuario, aplicando el principio de minimo privilegio.
- La directiva **WorkingDirectory** apunta a la carpeta del proyecto para que Python encuentre los módulos locales al importarlos.
- La directiva **ExecStart** especifica la ruta completa al interprete Python del entorno virtual, lo que garantiza que se usen exactamente las versiones de las librerias instaladas en ese entorno y no las del sistema.
- La directiva **Restart** con el valor *on-failure* indica que systemd debe reiniciar el proceso si termina con un código de error, con un intervalo de diez segundos entre reinicios definido por **RestartSec**.

```ini
[Unit]
Description=Bot Telegram DHT22
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=adrian
WorkingDirectory=/home/adrian/Documentos/SERVICIO-SOCIAL/REPORTES-SENSORES
```

```
ExecStart=/home/adrian/Documentos/SERVICIO-SOCIAL/REPORTES-SENSORES/venv/bin/python bot_telegram.py
Restart=on-failure
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Tras crear el archivo se ejecutaron los comandos:

- **sudo systemctl daemon-reload** para que systemd reconociera la nueva unidad.
- **sudo systemctl enable dht22bot.service** para activar el arranque automatico.
- **sudo systemctl start dht22bot.service** para iniciarlo de inmediato.

El estado del servicio se verifica con **systemctl status** y los registros de actividad se consultan con **journalctl -u dht22bot**, que es el equivalente a ver la salida de la terminal cuando el script se ejecuta manualmente.

REFERENCIAS

- Adafruit Industries. (2023). DHT sensor library for Arduino. GitHub. https://github.com/adafruit/DHT-sensor-library
- APScheduler Contributors. (2024). APScheduler documentation (Version 3.x). https://apscheduler.readthedocs.io/en/3.x/
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95. https://doi.org/10.1109/MCSE.2007.55
- McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56-61. https://doi.org/10.25080/Majora-92bf1922-00a
- Prometheus Authors. (2024). Prometheus HTTP API. https://prometheus.io/docs/prometheus/latest/querying/api/
- Prometheus Authors. (2024). Writing exporters. https://prometheus.io/docs/instrumenting/writing_exporters/
- python-telegram-bot Contributors. (2024). python-telegram-bot documentation (Version 21). https://python-telegram-bot.readthedocs.io/
- ReportLab Inc. (2024). ReportLab user guide: A PDF generation toolkit. https://www.reportlab.com/docs/reportlab-userguide.pdf
- Systemd Contributors. (2024). systemd.service: Service unit configuration. Freedesktop.org. https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html
- The Python Software Foundation. (2024). asyncio: Asynchronous I/O. Python 3.11 documentation. https://docs.python.org/3/library/asyncio.html
- The Python Software Foundation. (2024). statistics: Mathematical statistics functions. Python 3.11 documentation. https://docs.python.org/3/library/statistics.html